CS 486/686
Dissecting a Small Language Model

Yuntian Deng

Lecture 21

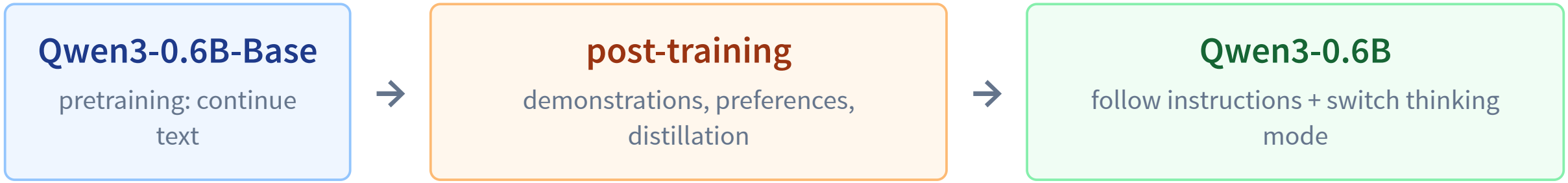
One real Qwen turn, end to end

Search › Uncertainty › Decisions › Learning

By the end, you can dissect one Qwen turn

- Run **Qwen3-0.6B** in Python or a browser.
- Trace messages → template → tokens → hidden states → logits.
- Map real Qwen config fields onto a transformer block.
- Explain **prefill, decode, and the KV cache**.
- Predict how decoding and prompts change behavior.
- Recognize where a fixed model is insufficient.

L20 built a base model. Today we run an assistant.



Same causal architecture; new behavior learned after pretraining.

Post-training turns a continuer into an answerer

Same prompt to both models: `What is gradient descent?`

base model (pretraining only)

How does it work? What is the role of the learning rate? How does the learning rate affect the convergence of the algorithm? What is the difference between gradient descent and stochastic gradient descent? …

continues with more question-like text

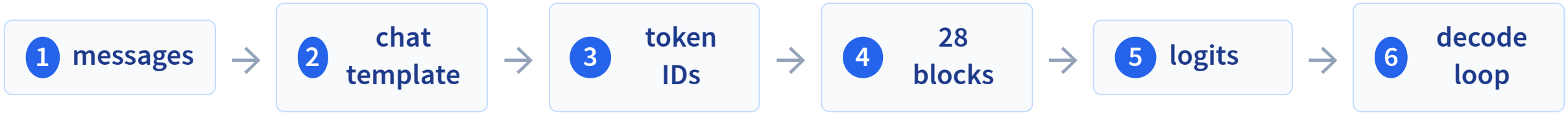
post-trained assistant

Gradient descent is a numerical method used in optimization to find the minimum (or maximum) of a function. It is commonly used in machine learning …

answers the question

Real Qwen3-0.6B-Base vs Qwen3-0.6B. Answering is *learned* from curated (instruction, answer) demonstrations and preference data.

One user turn, boundary by boundary



We will inspect the real object at every boundary.

The real model we will dissect

Qwen3-0.6B

dense, decoder-only, post-trained

28 layers	**1024** hidden width	**3072** FFN width	**16 / 8** query / KV heads*
128 head width	**151,936** vocabulary size	**32K** max training context	**tied** input/output matrix

* grouped-query attention — explained in a few slides.

Qwen Team, “Qwen3 Technical Report,” 2025.

Config is architecture written as data

```
{
  "hidden_size": 1024,
  "intermediate_size": 3072,
  "num_hidden_layers": 28,
  "num_attention_heads": 16,
  "num_key_value_heads": 8,
  "head_dim": 128,
  "vocab_size": 151936,
  "tie_word_embeddings": true
}
```

state	residual stream
communicate	16 query heads share 8 K/V heads
compute	3072-wide SwiGLU sublayer
predict	151,936 logits; input/output weights tied

The API receives structured messages

```
messages = [
    {
        "role": "user",
        "content": "Explain gradient descent in one senten
    }
]
```

role
who is speaking?

content
what did they say?

The model still consumes tokens—so this structure must become one string.

The chat template serializes one turn

```
<|im_start|>user
Explain gradient descent in one sentence.<|im_end|>
<|im_start|>assistant
generation starts here
```

`<|im_start|>`/`<|im_end|>` are ChatML message-boundary tokens (**im** = input message); each is one special token in the vocabulary.

No system message appears unless we actually supplied one.

Inspect the exact serialized input LIVE DATA

Single turn or multi-turn → Qwen special tokens → thinking ON/OFF generation cue.

`<|im_start|>user` message `<|im_end|>` `<|im_start|>assistant`

Thinking OFF pre-fills an empty think block; thinking ON lets Qwen generate the block.

Tokenization happens after templating

<|im_start|>	user	↵	Ex	plain	·gradient	·descent	·in	·one
151644	872	198	840	20772	20169	36952	304	825

·sentence	.
11652	13

… role markers + generation cue

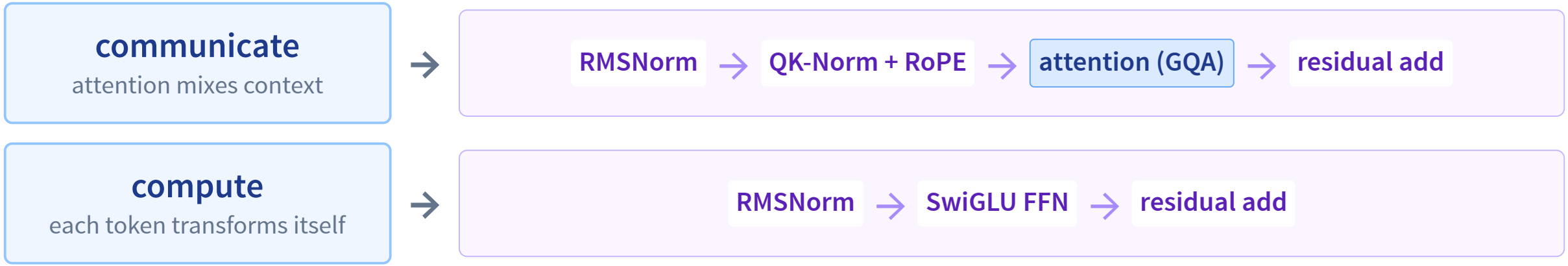
· means a leading space. IDs shown are actual Qwen tokenizer outputs, not illustrative.

Token IDs become a $[T, 1024]$ residual stream

Qwen position: RoPE rotates query/key dimensions inside attention; it is not an added position vector.

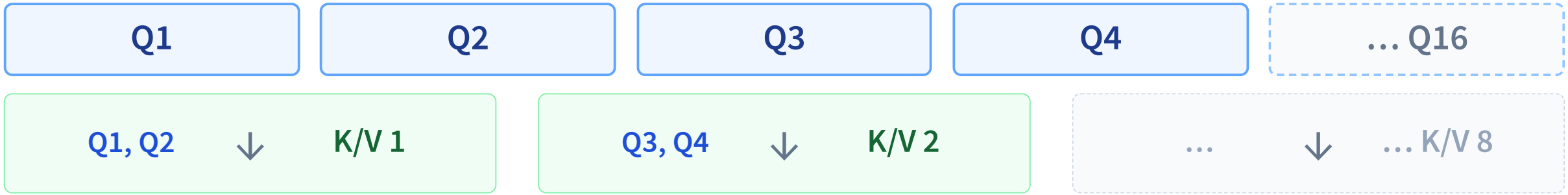
The same matrix E returns at the end to score output tokens.

L19’s block, with Qwen’s real component names



The abstraction is unchanged; these names specify Qwen’s implementation.

Grouped-query attention: 16 Q heads share 8 K/V heads



Why share? Keep many query views, but store half as many K/V heads in the cache.

The real model repeats this pattern: 16 query heads grouped over 8 key/value heads.

The same residual stream passes through 28 blocks

Every block adds an update; the residual stream carries the evolving $[T, 1024]$ representation.

Two heads read the same “it” differently

REAL MODEL DATA

Head A sends `it→cup`, head B sends `it→robot`. Switch to the unambiguous control to see what each head really tracks.

query: it | head A → cup 0.61 | head B → robot 0.75

On “The farmers loaded the truck because it was empty,” head B still finds the object (truck 0.84) while head A drifts to the start token. A curated head is a measurement, not an explanation.

Attention is a measurement—not an explanation

one head

among 16 heads × 28 layers

curated

selected because its pattern was interpretable

not causal proof

large weight does not prove decisive influence

Use attention to inspect a computation—not to claim the model's full reason.

Jain & Wallace, "Attention is not Explanation," NAACL 2019.

Wiegreffe & Pinter, "Attention is not not Explanation," EMNLP 2019.

One matrix does input and output

input embedding E

[vocab 151,936 × hidden 1024]

token ID → vector

same weights
TIED

output projection $E^\top$

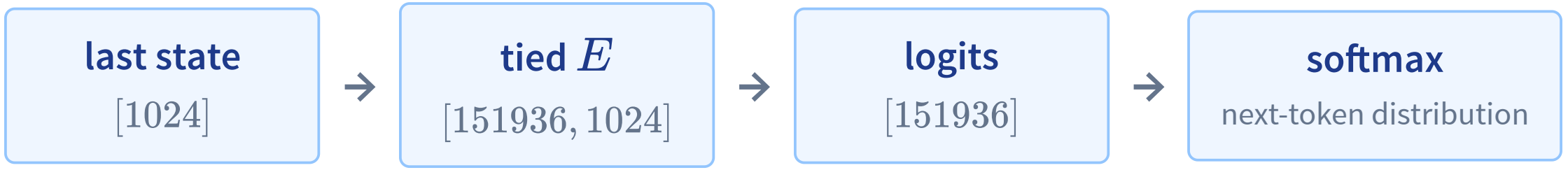
[hidden 1024 × vocab 151,936]

vector → one logit per vocab token

Each is **vocab × hidden = 151,936 × 1024 ≈ 156M parameters** (~0.3 GB).

That is about a quarter of a 0.6B model, so Qwen stores **one shared matrix**, not two.

The final state is scored by the tied embedding matrix



Gradient	0.985
The	0.004
A	0.004
all other tokens	~0.007

Real Qwen trace after the non-thinking template for “Explain gradient descent in one sentence.”

Append one token, then score again

REAL MODEL DATA

Step through an exact Qwen continuation; every distribution is conditioned on the enlarged context.

…assistant → Gradient → ·descent → ·is

Each appended token becomes part of the next model input.

Inference has two phases

1. prefill

prompt tokens in parallel

Build hidden states and K/V cache for the whole prompt.

2. decode

one new token

Generate sequentially, one distribution per step.

Parallel prefill; sequential decode.

Prefill: the prompt is already known

Every prompt token of `Explain gradient descent in one sentence.` is available up front.

`<|im_start|>` `user` `Ex` `plain` `·gradient` `·descent` `…` `assistant`

one parallel forward pass → hidden states + K/V for all prompt positions

Prefill = the single forward pass over the known prompt, done in parallel.

The KV cache reuses the past, one word at a time LIVE

Generate word by word: earlier hidden states and K/V are reused from cache, never recomputed.

prefill: compute all prompt states

each new word: reuse the past, compute 1 new column

Because attention is causal, earlier states never change, so decoding reuses them from the cache.

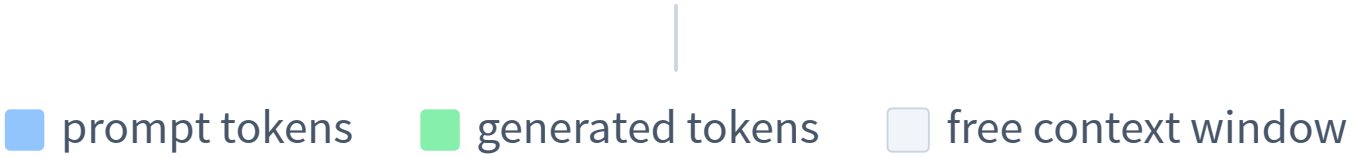
How much recomputation does the cache avoid? LIVE

One row per forward pass: with the cache each decode step adds a single new column.

with cache: $4 + 1 + 1 + 1 = 7$ without cache: $4 + 5 + 6 + 7 = 22$

Prefill computes the prompt once; each decode step then adds only one new cache column.

Context is working memory—and cache memory



window

prompt + output share Qwen's 32k context

cache memory

grows with layers × KV heads × length

latency

decode remains token-by-token

The cache buys speed by spending memory.

The complete Python inference path

```
# 1. load tokenizer + post-trained weights
tok = AutoTokenizer.from_pretrained("Qwen/Qwen3-0.6B")
model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen3-0.6B", device_map="auto")
# 2. serialize the role/content messages
text = tok.apply_chat_template(messages, add_generation_prompt=True, enable_thinking=False)
→ text[:34] = '<|im_start|>user\nExplain gradient…'
# 3. tokenize and move tensors beside the model
inputs = tok(text, return_tensors="pt").to(model.device)
→ inputs.input_ids.shape = [1, 20]
→ input_ids[0,:6] = [151644, 872, 198, 840, 20772, 20169]
# 4. autoregressively append new token IDs
all_ids = model.generate(**inputs, max_new_tokens=64)
→ all_ids.shape = [1, 48]  # 20 prompt + 28 new
# 5. drop the prompt IDs, then decode only the answer
answer = tok.decode(all_ids[0, 20:], skip_special_tokens=True)
→ answer = "Gradient descent is a method used to minimize a function…"
```

The same path can run in the browser

```
import { pipeline } from "@huggingface/transformers";

const qwen = await pipeline(
  "text-generation",
  "onnx-community/Qwen3-0.6B-ONNX",
  { device: "webgpu", dtype: "q4f16" },
);

const messages = [{
  role: "user",
  content: "Explain gradient descent.",
}];
const result = await qwen(messages, {
  max_new_tokens: 64,
  do_sample: true, temperature: 0.7, top_p: 0.8, top_k: 20,
});
```

instant path

audited traces embedded in the deck

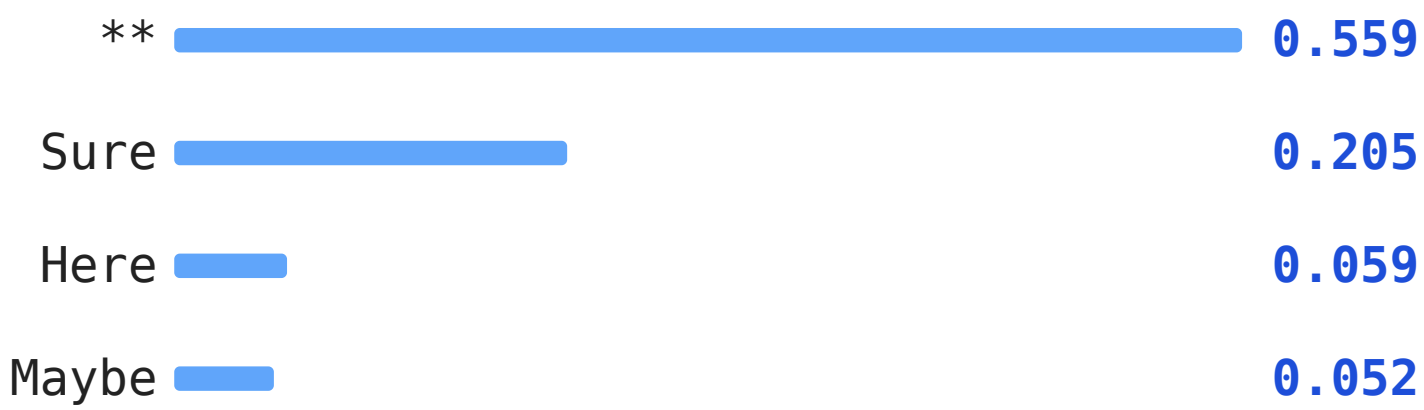
optional live path

several-hundred-MB download + compatible WebGPU

The concepts and default demos never depend on the live download succeeding.

One distribution, two selection rules

Qwen context: `Write a creative name for a friendly blue robot.`



greedy

always choose argmax: **

sampling

draw according to probability: varied path

The weights and distribution are fixed; only the selection rule changes.

Temperature reshapes; top-k and top-p trim

temperature	: sharper : unchanged : flatter	Ranking stays the same.
top-k = 3	keep **, Sure, Here	Remove every token below rank 3.
top-p = .8	.559 + .205 + .059 = .823	Keep the smallest prefix reaching 0.8.
renormalize	sample only from the kept candidates	

Run Qwen with recommended decoding presets

REAL MODEL DATA

Prompt `Explain gradient descent in one sentence.`
— each preset has its own exact distribution and continuation.

greedy | recommended non-thinking | recommended thinking

On the live slide, each preset changes the displayed distribution, chosen token, and continuation.

Longer generation: a math word problem REAL MODEL DATA

Non-thinking Qwen still lays out the steps over many tokens — step through it or reveal the full generation.

48 clips in April + 24 in May = 72

Qwen writes out 48 ÷ 2 = 24, then 48 + 24, and answers 72 clips.

Prompting changes tokens in context—not weights

role	task	audience	examples	format
You are a patient tutor.	Explain gradient descent.	Use language for a 10-year-old.	Input/output pairs specify a pattern.	One sentence; no equations.

zero-shot + examples = few-shot prompt

general prompt	“Gradient descent is a method used to minimize a function …”
10-year-old prompt	“Gradient descent is like a way to get closer to the best answer …”

Real greedy Qwen outputs. More instructions and examples are simply more conditioning tokens.

Thinking mode changes the generation protocol

thinking OFF

```
<|im_start|>assistant
<think>

</think>
```

Template pre-fills an empty **<think></think>** block, so Qwen goes straight to the answer.

thinking ON

```
<|im_start|>assistant
```

Template stops at the cue, so Qwen generates **<think>…</think>** before the answer.

Reasoning text can help on hard tasks, but costs tokens and is not guaranteed correct or causally faithful.

A fluent answer can still be fabricated REAL MODEL DATA

Ask about the future and watch a confident, invented answer
generate token by token — then reveal the full fabricated name.

Who won the 2031 Turing Award? →

"…the 2031 Turing Award was awarded to **Amitabh Patel** for his work in machine learning."

2031 is in the future; the correct behavior is to abstain or retrieve current evidence.

What changed today—and what did not

changed today	did not change
chat template, context, decoding rule	weights, stored knowledge, available actions

We steered a fixed model through its inputs and its decoding—nothing more.

Recap

- Messages → chat template → exact tokens → $[T, 1024]$ states.
- Qwen block: RMSNorm, RoPE + QK-Norm, grouped-query attention, SwiGLU.
- Tied E scores ~152k logits; softmax gives the next-token distribution.
- Prefill in parallel, decode sequentially, reuse the KV cache.
- Decoding and prompts steer behavior; the weights stay fixed.

L22: when inputs are not enough—prompting vs RAG/tools vs SFT/LoRA, and Qwen as a frozen interpreter for PAW.